Center Star Multiple Sequence Alignment Tool - Python

Weronika Łaszkiewicz, 278434

21.05.2025

1 Introduction

The Center Star algorithm is a widely used heuristic for performing multiple sequence alignment (MSA), particularly effective when working with three or more biological sequences. In this project, I implemented the algorithm in Python and supplemented it with statistical analysis and visualization components. The tool supports both DNA and protein sequence alignment and provides quantitative metrics to assess alignment quality.

The complete source code and documentation are available in the project's GitHub repository:
github.com/wlaszkiewicz/center-star-msa

2 System Design

2.1 Project Structure

```
center_star_msa/
├── main.py ..................................Main application entry point
├── core/
│   ├── __init__.py
│   ├── alignment.py .........Needleman-Wunsch, CenterStar alignment logic
│   ├── consensus.py ..............................Consensus generation
│   ├── statistics.py ....................................MSA statistics
│   └── fasta_parser.py ..............................Fasta files parser
└── gui/
    ├── __init__.py
    ├── main_window.py ..............................MainWindow class
    └── dialogs.py ........................................Input dialogs
```

3 Algorithm Implementation Details

3.1 Mathematical Foundation

The Center Star algorithm solves the MSA problem through a series of pairwise alignments centered around the most representative sequence. The key steps involve:

1. Selecting the center sequence S_c that maximizes the sum of pairwise alignment scores against all other sequences S_i:

$$\sum_{i=1}^{n} \text{score}(S_c, S_i) \tag{1}$$

   where $\text{score}(S_c, S_i)$ is obtained from a Needleman-Wunsch global alignment algorithm.

2. Initializing the multiple sequence alignment (MSA) with this center sequence S_c.

3. Progressively aligning all other sequences S_i one by one to the current version of S_c (which accumulates gaps from previous alignments).

4. After each pairwise alignment of S_i to S_c, the new gaps introduced into S_c must be propagated to all other sequences already incorporated into the MSA to maintain alignment consistency.

3.2 Algorithm Overview

The implementation follows the Center Star approach with four main phases:

1. **Center Selection**: Computes all pairwise alignment scores using Needleman-Wunsch to identify the optimal center sequence (the one with the maximum total score when aligned against all others).

2. **MSA Initialization**: Creates an initial MSA profile containing only the selected center sequence.

3. **Progressive Alignment**: Iteratively aligns each remaining sequence to the evolving center sequence using algorithm.

4. **Gap Propagation**: After each sequence is aligned to the center sequence, any new gaps inserted into the center sequence are propagated to all other sequences already in the MSA.

3.3 Core Algorithm

The key components are implemented across several functions:

Algorithm 1 Center Star MSA

```
Compute all pairwise alignment scores between all sequences.
Select the sequence S_c with the maximum total sum of pairwise scores as the center sequence.
Initialize MSA with S_c.
for each remaining sequence S_i do
    Align S_i to the current version of S_c in the MSA using Needleman-Wunsch. This may introduce new gaps into S_c and S_i.
    Update S_c in the MSA with its newly gapped version.
    Propagate any new gaps introduced in S_c (from the alignment with S_i) to all other sequences already aligned in the MSA.
    Add the aligned S_i (with gaps corresponding to S_c's current state) to the MSA.
end for
Generate consensus sequence from the final MSA.
Compute alignment statistics.
```

3.4 Key Features

- Support for DNA/protein sequences.
- Configurable scoring parameters for Needleman-Wunsch:
  - Match score (default: +1)
  - Mismatch penalty (default: -1)
  - Gap penalty (default: -2)
- Statistics:
  - **Average pairwise identity:** The average of all individual pairwise sequence identity percentages from the alignment.
  - **Fully conserved columns:** A count of alignment columns where every sequence has the exact same character (not a gap).
  - **Sum-of-pairs score (SP Score):** The total sum of alignment scores (match/mismatch/gap) from all unique pairs of sequences in the MSA.
- Visualization outputs:
  - **Center election matrix:** An $N \times N$ matrix where cell (i, j) is the Needleman-Wunsch score between original sequence i and j.
  - **Pairwise identity/distance matrices:** $N \times N$ matrices from the final alignment:
    - The *identity matrix*: Cell (i, j) is the percentage of identical characters between aligned sequence i and j.
    - The *distance matrix*: Cell (i, j) is 100%−(identity % between aligned i and j).

3.5 Key Methods

The core logic is primarily implemented in the following Python functions:

3.5.1 Needleman-Wunsch Global Alignment (needleman_wunsch)

This function performs global alignment between two sequences. It is adapted to handle sequences that may already contain gap characters ('-'), which is important when aligning a sequence to an evolving center profile that has accumulated gaps.

```python
def needleman_wunsch(seq1_str: str, seq2_str: str, match_score: int = 1, mismatch_penalty: int = -1,
                     gap_penalty: int = -1) -> tuple[float, str, str]:

    seq1, seq2 = list(seq1_str), list(seq2_str)
    n, m = len(seq1), len(seq2)
    dp = np.zeros((n + 1, m + 1))

    # initializing DP table
    for i in range(n + 1):
        dp[i][0] = i * gap_penalty
    for j in range(m + 1):
        dp[0][j] = j * gap_penalty

    # filling DP table
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            char1, char2 = seq1[i - 1], seq2[j - 1]

            current_match_mismatch_score = 0
            if char1 == char2 and char1 != '-':  # match
                current_match_mismatch_score = match_score
            elif char1 == '-' and char2 == '-':  # both are gaps already (can happen if aligning profiles)
                current_match_mismatch_score = 0  # or some other neutral/gap extension penalty
            elif char1 == '-' or char2 == '-':  # one is a gap, align with gap penalty
                current_match_mismatch_score = gap_penalty
            else:  # mismatch
                current_match_mismatch_score = mismatch_penalty

            score_diag = dp[i - 1][j - 1] + current_match_mismatch_score
            score_up = dp[i - 1][j] + gap_penalty
            score_left = dp[i][j - 1] + gap_penalty
            dp[i][j] = max(score_diag, score_up, score_left)

    #traceback
    aligned_s1, aligned_s2 = [], []
    i_tb, j_tb = n, m
    score = dp[n][m]

    while i_tb > 0 or j_tb > 0:
        current_score_val = dp[i_tb][j_tb]
        s1_char_tb = seq1[i_tb - 1] if i_tb > 0 else ''
        s2_char_tb = seq2[j_tb - 1] if j_tb > 0 else ''

        diag_path_score_component = 0
        if i_tb > 0 and j_tb > 0:
            if s1_char_tb == s2_char_tb and s1_char_tb != '-':
                diag_path_score_component = match_score
            elif s1_char_tb == '-' and s2_char_tb == '-':
                diag_path_score_component = 0
            elif s1_char_tb == '-' or s2_char_tb == '-':
                diag_path_score_component = gap_penalty
            else:  # Mismatch
                diag_path_score_component = mismatch_penalty

        # check diagonal path (match/mismatch)
        if i_tb > 0 and j_tb > 0 and abs(
                current_score_val - (dp[i_tb - 1][j_tb - 1] + diag_path_score_component)) < 1e-9:
            aligned_s1.append(s1_char_tb)
            aligned_s2.append(s2_char_tb)
            i_tb -= 1
            j_tb -= 1
        # check up path (gap in seq2)
```

```python
        elif i_tb > 0 and abs(current_score_val - (dp[i_tb - 1][j_tb] + gap_penalty)) < 1e-9:
            aligned_s1.append(s1_char_tb)
            aligned_s2.append('-')
            i_tb -= 1
        # check left path (gap in seq1)
        elif j_tb > 0 and abs(current_score_val - (dp[i_tb][j_tb - 1] + gap_penalty)) < 1e-9:
            aligned_s1.append('-')
            aligned_s2.append(s2_char_tb)
            j_tb -= 1
        else:  # fallback
            if i_tb > 0:  # prefer consuming from seq1 if ambiguous
                aligned_s1.append(s1_char_tb)
                aligned_s2.append('-')
                i_tb -= 1
            elif j_tb > 0:
                aligned_s1.append('-')
                aligned_s2.append(s2_char_tb)
                j_tb -= 1
            else:  # both i_tb and j_tb are 0
                break
    return score, "".join(reversed(aligned_s1)), "".join(reversed(aligned_s2))
```

3.5.2 Center Sequence Determination (get_center_sequence)

Identifies the optimal center sequence by finding the sequence that has the highest sum of alignment scores against all other sequences.

```python
def get_center_sequence(sequences_with_ids: list[tuple[str, str]], match_score: int = 1,
                        mismatch_penalty: int = -1, gap_penalty: int = -1) -> tuple[
    int, tuple[str, str] | None, list[tuple[str, str]], list[tuple[str, str]], list[list[float]],
    ↪  list[float]]:

    if not sequences_with_ids:
        return -1, None, [], sequences_with_ids, [[]], []

    num_sequences = len(sequences_with_ids)
    seq_strings_only = [item[1] for item in sequences_with_ids]

    if num_sequences == 1:
        return 0, sequences_with_ids[0], [], sequences_with_ids, [[0.0]], [0.0]

    pairwise_scores_matrix = [[0.0] * num_sequences for _ in range(num_sequences)]
    total_scores_for_center_candidate = [0.0] * num_sequences
    max_total_score = -float('inf')
    center_idx = -1

    for i in range(num_sequences):
        current_i_total_score = 0.0
        for j in range(num_sequences):
            if i == j:
                pairwise_scores_matrix[i][j] = 0.0  # score of a sequence with itself
                continue
            if j < i:  # already computed score
                score_ij = pairwise_scores_matrix[j][i]
            else:
                score_ij, _, _ = needleman_wunsch(seq_strings_only[i], seq_strings_only[j],
                ↪  match_score,
                                                  mismatch_penalty, gap_penalty)
            pairwise_scores_matrix[i][j] = score_ij
            if j > i: pairwise_scores_matrix[j][i] = score_ij
            current_i_total_score += score_ij
        total_scores_for_center_candidate[i] = current_i_total_score
        if current_i_total_score > max_total_score:
            max_total_score = current_i_total_score
            center_idx = i

    if center_idx == -1 and num_sequences > 0:  # always find a center if sequences exist
        center_idx = 0  # default to first sequence if scores are all equal or problematic

    center_item_tuple = sequences_with_ids[center_idx]
    other_items = [item for idx, item in enumerate(sequences_with_ids) if idx != center_idx]

    return center_idx, center_item_tuple, other_items, sequences_with_ids, pairwise_scores_matrix,
    ↪  total_scores_for_center_candidate
```

3.5.3 Progressive Alignment (center_star)

Builds the multiple sequence alignment. After the center sequence is determined, it's placed in the MSA. Then, each of the "other" sequences is aligned to the current version of the center sequence (which may already contain gaps from previous alignments). After each such alignment, the center sequence itself is updated in the MSA. Any new gaps introduced into the center sequence must be propagated to all other sequences that have already been added to the MSA. This is handled by an internal loop that iterates through the characters of the newly modified center sequence and compares them to the center sequence's state before the latest alignment, inserting gaps into the other MSA rows.

```python
def center_star(original_sequences_with_ids: list[tuple[str, str]], match_score: int = 1, mismatch_penalty: int = -1,
                gap_penalty: int = -1) -> list[str]:

    if not original_sequences_with_ids:
        return []

    num_input_seqs = len(original_sequences_with_ids)
    if num_input_seqs == 1:
        return [original_sequences_with_ids[0][1]]  # return the single sequence as is

    # 1. determine the center sequence
    center_original_idx_internal, center_item_internal, other_items_internal, _, _, _ = \
        get_center_sequence(original_sequences_with_ids, match_score, mismatch_penalty, gap_penalty)

    if center_item_internal is None:  # Should not happen if input is not empty
        return [""] * num_input_seqs

    # initialize MSA with the center sequence
    center_seq_str_current_msa = list(center_item_internal[1])
    msa_sequences_list = [center_seq_str_current_msa]  # list of lists of characters

    # 2. align each other sequence to the current MSA (evolving center sequence)
    for other_id, other_seq_raw in other_items_internal:
        center_profile_str = "".join(msa_sequences_list[0])  # current state of the center sequence in MSA

        # align the new sequence to the current center profile
        _, aligned_center_profile, aligned_new_other = needleman_wunsch(
            center_profile_str, other_seq_raw, match_score, mismatch_penalty, gap_penalty
        )

        # update the MSA:
        msa_sequences_list[0] = list(aligned_center_profile)

        # propagate gaps from the new center profile to all other sequences already in MSA
        for k in range(1, len(msa_sequences_list)):  # for each existing non-center sequence in MSA
            current_other_seq_in_msa = msa_sequences_list[k]
            updated_other_seq = []
            old_profile_char_idx = 0

            for new_profile_char in aligned_center_profile:  # iterate through new center
                if old_profile_char_idx < len(center_profile_str) and \
                        new_profile_char == center_profile_str[old_profile_char_idx]:
                    # Char in center matches old center char, copy from existing aligned sequence
                    updated_other_seq.append(current_other_seq_in_msa[old_profile_char_idx])
                    old_profile_char_idx += 1
                else:
                    # gap introduced in center profile, or character changed

                    updated_other_seq.append('-')
                    # the new_profile_char is NOT a gap - a character from other_seq_raw was aligned there.

                    if new_profile_char != '-' and \
                            old_profile_char_idx < len(center_profile_str) and \
                            center_profile_str[old_profile_char_idx] != '-':
                        old_profile_char_idx += 1

            msa_sequences_list[k] = updated_other_seq
```

```python

        # newly aligned sequence to the MSA
        msa_sequences_list.append(list(aligned_new_other))

    # 3. reconstruct the final MSA strings in the original input order
    final_ordered_msa_strings = [""] * num_input_seqs
    if not msa_sequences_list or not msa_sequences_list[0]:  # MSA is empty
        return final_ordered_msa_strings

    # placing the center sequence in its original position
    final_ordered_msa_strings[center_original_idx_internal] = "".join(msa_sequences_list[0])

    # Placing the other (now aligned) sequences in their original positions
    other_seq_msa_idx = 1

    for i in range(num_input_seqs):
        if i == center_original_idx_internal:
            continue  # already placed
        if other_seq_msa_idx < len(msa_sequences_list):
            final_ordered_msa_strings[i] = "".join(msa_sequences_list[other_seq_msa_idx])
            other_seq_msa_idx += 1
        else:
            # this should not be hit if logic is correct

                final_ordered_msa_strings[i] = "-" * len(msa_sequences_list[0]) if msa_sequences_list and msa_sequences_list[0] else ""

    return final_ordered_msa_strings
```

3.5.4 Consensus Generation (generate_consensus)

Creates a consensus sequence from the final multiple sequence alignment. For each column, if the proportion of gaps meets or exceeds a defined threshold, a gap is placed in the consensus. Otherwise, the most frequent non-gap character is chosen. Ties between non-gap characters are resolved alphabetically.

```python
from collections import Counter

def generate_consensus(aligned_sequences: list[str], gap_threshold: float = 0.51) -> str:

    if not aligned_sequences or not aligned_sequences[0]:
        return ""

    num_seqs = len(aligned_sequences)
    aln_len = len(aligned_sequences[0])
    consensus_seq = []

    for j in range(aln_len):  #  columns
        column_chars = [aligned_sequences[i][j] for i in range(num_seqs)]
        counts = Counter(column_chars)

        num_gaps_in_col = counts.get('-', 0)

        #if gap frequency is above threshold, consensus is a gap
        if num_seqs > 0 and (num_gaps_in_col / num_seqs) >= gap_threshold:
            consensus_seq.append('-')
            continue

        #the most frequent non-gap character
        most_common_non_gap = []
        max_freq_non_gap = 0

        sorted_chars = sorted(counts.keys())

        for char in sorted_chars:
            if char != '-':
                freq = counts[char]
                if freq > max_freq_non_gap:
                    max_freq_non_gap = freq
                    most_common_non_gap = [char]
                elif freq == max_freq_non_gap:
                    most_common_non_gap.append(char)

```

```
        if most_common_non_gap:
            # alphabetical tie-breaking is handled by sorted_chars

            most_common_non_gap.sort()
            consensus_seq.append(most_common_non_gap[0])
        else:
            # all characters in the column were gaps, but didn't meet gap_threshold
            # this shouldnt ideally not happen if gap_threshold is e.g. >0.5 and there's at least one non-gap,
            consensus_seq.append('-')

    return "".join(consensus_seq)
```

3.6 Complexity Analysis

The implementation demonstrates the following theoretical complexities, where N is the number of sequences, $L_{initial}$ is the average initial length of these sequences, and L_{final} is the length of the final multiple sequence alignment. It's important to note that L_{final} can be significantly larger than $L_{initial}$, potentially up to $O(N \cdot L_{initial})$ in worst-case scenarios.

Operation	**Time**	**Space**
Pairwise Comparisons (for Center Selection)	$O(N^2 \cdot L_{initial}^2)$	$O(N^2 + L_{initial}^2)$
Progressive Alignment	$O(NL_{initial}L_{final} + N^2 L_{final})$	$O(N \cdot L_{final})$
Consensus Generation	$O(N \cdot L_{final})$	$O(L_{final})$

Table 1: Simplified computational complexity by operation.

Pairwise Comparisons (for Center Selection):

To find the best center sequence, approximately $N^2/2$ pairwise global alignments are performed (Needleman-Wunsch). Each alignment between two sequences of average length $L_{initial}$ takes $O(L_{initial}^2)$ time. Space is required to store the pairwise scores (an $N \times N$ matrix, so $O(N^2)$) and the dynamic programming table for a single alignment ($O(L_{initial}^2)$).

Progressive Alignment:

This involves $N - 1$ iterations, one for each non-center sequence. In each iteration:

- A sequence (average length $L_{initial}$) is aligned to the evolving center profile. As the center profile can grow up to length L_{final}, this alignment takes roughly $O(L_{initial} \cdot L_{final})$ time.
- Gaps introduced into the center sequence are then propagated to the other $O(N)$ sequences already in the alignment, across the length L_{final}. This step takes about $O(N \cdot L_{final})$ time.

These operations are repeated $N - 1$ times so the total time complexity for progressive alignment is approximately $O(N \cdot L_{initial} \cdot L_{final} + N^2 \cdot L_{final})$. The space requirement is for storing the final MSA, which has N sequences of length L_{final}, leading to $O(N \cdot L_{final})$.

Consensus Generation:

To generate the consensus sequence, each of the L_{final} columns in the completed MSA is examined. For every column, all N sequences are checked to determine the consensus character. This results in a time complexity of $O(N \cdot L_{final})$. The space needed is primarily for the consensus string itself, which is $O(L_{final})$.

Optimality:

The Center Star algorithm is a *heuristic* approach. It is computationally more efficient than methods that attempt to find the mathematically optimal multiple sequence alignment but it does not guarantee that the resulting alignment will be the optimal one (e.g., the one with the absolute highest possible sum-of-pairs score). The quality of the alignment produced can depend on the initial choice of the center sequence and the order in which other sequences are added to the alignment.

4 Example Results

4.1 Alignment Results

```
--- Alignment Parameters Used ---
  - Match Score: 1
  - Mismatch Penalty: -1
  - Gap Penalty: -2

-> Chosen Center Sequence (for MSA): s1_test (Sum of pairwise scores: 1.0)
   (See 'Center Election Matrix' tab for details)


--- Aligned Sequences & Per-Sequence Stats ---
The sequence 's1_test' (marked with '*' below) was used as the initial center.

  ID            Aligned Sequence   Gaps     MatchCons    MismatchCons    ID%Cons
  ---------------------------------------------------------------------------------
Consensus     AT-TGCCATT    1       N/A         N/A            100.0%
  ---------------------------------------------------------------------------------
* s1_test     AT-TGCCATT     1        9           0              100.00  %
  s2_test     AT-GGCCATT     1        8           1              88.89   %
  s3_test     ATCCATTTTT     0        4           5              44.44   %
  s4_test     ATCT-TC-TT     2        6           1              66.67   %
  s5_test     AC-TG--ACC     3        4           3              44.44   %

Per-Sequence Stats Explanation (vs Consensus):
  - Gaps: Number of '-' characters in this specific aligned sequence.
  - MatchCons: Number of characters matching 'Consensus' (Consensus not gap).
  - MismatchCons: Number of characters differing from 'Consensus' (neither are gaps
    , Consensus not gap).
  - ID%Cons: Percent identity to 'Consensus' (Consensus not gap positions).


--- Overall MSA Summary Statistics ---
  - Number of Sequences in MSA: 5
  - Alignment Length (columns): 10
  - Average Pairwise Identity (PID): 44.67%
  - Total Gaps in MSA: 7
  - Fully Conserved Columns (non-gap): 2
  - Sum-of-Pairs Score (SP Score): -33.0
  - Generated Consensus Sequence Length: 10
```

4.2 Matrix Visualizations

Alignment Summary | Center Election Matrix | Pairwise Identity Matrix (%) | Pairwise Distan

	s1_test	s2_test	s3_test	s4_test	s5_test	Sum
s1_test	-	7.0	-3.0	0.0	-3.0	**1.0**
s2_test	7.0	-	-3.0	0.0	-4.0	0.0
s3_test	-3.0	-3.0	-	2.0	-7.0	-11.0
s4_test	0.0	0.0	2.0	-	-3.0	-1.0
s5_test	-3.0	-4.0	-7.0	-3.0	-	-17.0

Figure 1: Center election matrix visualization

Alignment Summary | Center Election Matrix | Pairwise Identity Matrix (%) | Pairwise Distan

	s1_test	s2_test	s3_test	s4_test	s5_test
s1_test	100.00	88.89	40.00	60.00	44.44
s2_test	88.89	100.00	40.00	50.00	33.33
s3_test	40.00	40.00	100.00	60.00	10.00
s4_test	60.00	50.00	60.00	100.00	20.00
s5_test	44.44	33.33	10.00	20.00	100.00

Figure 2: Pairwise identity matrix (percentage)

nent Summary | Center Election Matrix | Pairwise Identity Matrix (%) | Pairwise Distance Matrix

	s1_test	s2_test	s3_test	s4_test	s5_test
s1_test	0.00	11.11	60.00	40.00	55.56
s2_test	11.11	0.00	60.00	50.00	66.67
s3_test	60.00	60.00	0.00	40.00	90.00
s4_test	40.00	50.00	40.00	0.00	80.00
s5_test	55.56	66.67	90.00	80.00	0.00

Figure 3: Pairwise distance matrix

5 Evolutionary Comparison of Example Sequences

For a consistent and fair comparison, all alignments were generated using the following parameters:

- Match score: +1
- Mismatch penalty: -1
- Gap penalty: -2

All protein sequences were retrieved from the NCBI GenBank database (`https://www.ncbi.nlm.nih.gov/genbank/`). .

5.1 Related Sequences Analysis: Cytochrome C

For this analysis, I selected the protein sequences of Cytochrome C from four vertebrate species evolutionarily related. Protein sequences were chosen because of their more conserved nature over longer evolutionary timescales compared to nucleotide sequences and because of the use of the sequence length for practical alignment. The selected species and their NCBI GenBank accession numbers are:

- Human (*Homo sapiens*): NP_061820.1
- Mouse (*Mus musculus*): CAA25899.1
- Chicken (*Gallus gallus*): NP_001385227.1
- Zebrafish (*Danio rerio*): NP_001002068.1

5.1.1 Alignment Output and Statistics

The following output was generated by the MSA tool:

```
--- Alignment Parameters Used ---
Match Score: 1, Mismatch Penalty: -1, Gap Penalty: -2


--- Center Election Score Matrix ---
This matrix shows the alignment scores between all pairs of your input sequences.
    The Center Star algorithm picks the sequence with the highest 'Sum' score as
    the 'center' to guide the multiple alignment.

             NP_0010020  NP_0013852  CAA25899.1  NP_061820.  | Sum
----------------------------------------------------
NP_0010020   -           80.0        84.0        70.0        | 234.0
NP_0013852   80.0        -           89.0        79.0        | 248.0
CAA25899.1   84.0        89.0        -           87.0        | 260.0
NP_061820.   70.0        79.0        87.0        -           | 236.0

Chosen Center Sequence: CAA25899.1 (Total Sum Score: 260.0)


--- Overall MSA Summary Statistics ---
Number of Sequences in MSA: 4
Alignment Length: 105
Average Pairwise Identity: 88.89%
Total Gaps in MSA: 1
Fully Conserved Columns: 86
Sum-of-Pairs Score: 487.0
Generated Consensus Length: 105


--- Pairwise Identity Matrix (\%) ---
             NP_0010020  NP_0013852  CAA25899.1  NP_061820.
```

```
------------------------------------------------------------
NP_0010020    100.00       87.62        90.48        83.81
NP_0013852    87.62        100.00       92.38        87.62
CAA25899.1    90.48        92.38        100.00       91.43
NP_061820.    83.81        87.62        91.43        100.00


--- Pairwise Distance Matrix ---
              NP_0010020   NP_0013852   CAA25899.1   NP_061820.
------------------------------------------------------------
NP_0010020    0.00         12.38        9.52         16.19
NP_0013852    12.38        0.00         7.62         12.38
CAA25899.1    9.52         7.62         0.00         8.57
NP_061820.    16.19        12.38        8.57         0.00
```

Listing 1: MSA Tool Output for Related Cytochrome C Sequences

The actual alignment of the sequences, along with per-sequence statistics against the consensus, is shown in Figure 4. The visualization matrices providing more insight are presented in Figure 5.

```
--- Aligned Sequences & Per-Sequence Stats ---
The sequence 'CAA25899.1' (marked with '* ' below) was used as the initial center.

  ID            Aligned Sequence                                                
```

- **Gaps:** Only 1 total gap appears in the entire MSA (Listing 1), indicating the sequences align very well without many insertions or deletions, further supporting their homology.
- **Alignment Score:** The Sum-of-Pairs Score is 487.0 (Listing 1), a high value reflecting a well-matched alignment.

In summary, the Center Star tool successfully aligned these homologous Cytochrome C sequences, resulting in statistics (high identity, strong conservation, few gaps) and matrix visualizations that clearly demonstrate their close evolutionary relationship.

5.2 Unrelated Sequences Analysis

To contrast with the homologous sequences, a set of four evolutionarily unrelated proteins with diverse functions and origins was selected. The aim is to observe how the Center Star MSA tool handles sequences lacking significant shared ancestry. The selected proteins and their NCBI GenBank accession numbers are:

- Human (*Homo sapiens*) Cytochrome C: NP_061820.1
- Human (*Homo sapiens*) Thioredoxin (TXN): CAG28593.1
- Zebrafish (*Danio rerio*) Myoglobin (mb): AAH65862.1
- E. coli (*Escherichia coli*) Lac Repressor (LacI): QCB65867.1

5.2.1 Alignment Output and Statistics

The MSA tool generated the following output for these unrelated sequences:

```
--- Alignment Parameters Used ---
Match Score: 1, Mismatch Penalty: -1, Gap Penalty: -2


--- Center Election Score Matrix ---
This matrix shows the alignment scores between all pairs of your input sequences.
    The Center Star algorithm picks the sequence with the highest 'Sum' score as
    the 'center' to guide the multiple alignment.

             NP_061820.  CAG28593.1  AAH65862.1  QCB65867.1  | Sum
-----------------------------------------------------
NP_061820.   -           -75.0       -129.0      -497.0      | -701.0
CAG28593.1   -75.0       -           -133.0      -495.0      | -703.0
AAH65862.1   -129.0      -133.0      -           -429.0      | -691.0
QCB65867.1   -497.0      -495.0      -429.0      -           | -1421.0

Chosen Center Sequence: AAH65862.1 (Total Sum Score: -691.0)


--- Overall MSA Summary Statistics ---
Number of Sequences in MSA: 4
Alignment Length: 360
Average Pairwise Identity: 12.29\%
Total Gaps in MSA: 723
Fully Conserved Columns: 231
Sum-of-Pairs Score: -2039.0
Generated Consensus Length: 360


--- Pairwise Identity Matrix (\%) ---
             NP_061820.  CAG28593.1  AAH65862.1  QCB65867.1
-------------------------------------------------------
NP_061820.   100.00      5.15        20.41       5.00
CAG28593.1   5.15        100.00      19.05       4.17
```

```
AAH65862.1    20.41        19.05        100.00       20.00
QCB65867.1    5.00         4.17         20.00        100.00


--- Pairwise Distance Matrix ---
              NP_061820.   CAG28593.1   AAH65862.1   QCB65867.1
-------------------------------------------------------------
NP_061820.    0.00         94.85        79.59        95.00
CAG28593.1    94.85        0.00         80.95        95.83
AAH65862.1    79.59        80.95        0.00         80.00
QCB65867.1    95.00        95.83        80.00        0.00
```

Listing 2: MSA Tool Output for Unrelated Protein Sequences

The visual representation of the forced alignment and per-sequence statistics is anticipated to show extensive gapping (Figure 6). The visualization matrices are shown in Figure 7.

```
--- Aligned Sequences & Per-Sequence Stats ---
The sequence 'AAH65862.1' (marked with '* ' below) was used as the initial center.

  ID            Aligned Sequence
  ---------------------------------------------------------------------------------
Consensus      MADGDL--VAKCAG--------AV-EA-DKAAKFCE-V------L----NR----L-GK----AAGDT--LKL-
  ---------------------------------------------------------------------------------
  NP_061820.   M--GD---VEK--G--------KK-IF-IMKCSQCH-T------V----EK----G-GK-----HKTG--PNL-
  CAG28593.1   M-------V-K-----------QI-E--SKTA-FQE--------------A----L--D----AAGD----KL--
* AAH65862.1   MADHDL--VLKCWG--------AV-EA-DYAANGGE-V------L----NR----L-FK----EYPDT--LKL-
  QCB65867.1   MKPVTLYDVAEYAGVSYQTVSRVVNQASHVSAKTREKVEAAMAELNYIPNRVAQQLAGKQSLLIGVATSSLALH
```

(a) Beginning portion of the multiple sequence alignment for unrelated sequences.

```
                                                  Gaps    MatchCons   MismatchCons   ID%Cons
-----------------------------------------------------------------------------------------
----YEK------------------EIAEA--------G-   213    N/A        N/A           100.0%
-----------------------------------------------------------------------------------------
----YLK--------------------KATN--------E-    255     36          69             24.49   %
----LEA------------------TINEL--------V-    255     44          61             29.93   %
----YYK------------------EIGFA--------G-    213     116         31             78.91   %
SLVKRKTTLAPNTQTASPRALADSLMQLARQVSRLESGQ    0       84          63             57.14   %
```

(b) Ending portion of the multiple sequence alignment for unrelated sequences.

Figure 6: Beginning (a) and ending (b) portions of the alignment for four unrelated protein sequences. Extensive gapping is expected throughout the alignment due to the lack of homology.

	NP_061820.	CAG28593.1	AAH65862.1	QCB65867.1	Sum
NP_061820.	-	-75.0	-129.0	-497.0	-701.0
CAG28593.1	-75.0	-	-133.0	-495.0	-703.0
AAH65862.1	-129.0	-133.0	-	-429.0	**-691.0**
QCB65867.1	-497.0	-495.0	-429.0	-	-1421.0

(a) Center Election

	NP_061820.	CAG28593.1	AAH65862.1	QCB65867.1
NP_061820.	100.00	5.15	20.41	5.00
CAG28593.1	5.15	100.00	19.05	4.17
AAH65862.1	20.41	19.05	100.00	20.00
QCB65867.1	5.00	4.17	20.00	100.00

(b) Pairwise Identity

	NP_061820.	CAG28593.1	AAH65862.1	QCB65867.1
NP_061820.	0.00	94.85	79.59	95.00
CAG28593.1	94.85	0.00	80.95	95.83
AAH65862.1	79.59	80.95	0.00	80.00
QCB65867.1	95.00	95.83	80.00	0.00

(c) Pairwise Distance

Figure 7: Visualization matrices for the alignment of four unrelated protein sequences.

5.2.2 Analysis of Results

The alignment of these four diverse and evolutionarily distant proteins shows results that contrast with the homologous Cytochrome C set, as expected for unrelated sequences:

- **Center Sequence & Scores:** Zebrafish Myoglobin (AAH65862.1) was selected as the center sequence. However, all pairwise alignment scores and the resulting sum scores in the Center Election Matrix (Listing 2, Figure 7a) are highly negative (e.g., a sum score of -691.0 for the chosen center). This indicates no genuinely good "central" sequence exists among these unrelated proteins.
- **Similarity:** An Average Pairwise Identity of only 12.29% (Listing 2) confirms significant dissimilarity. The Pairwise Identity Matrix (Figure 7b) shows that most individual pairwise identities are very low (typically 4-20%), consistent with a lack of shared evolutionary origin.
- **Forced Alignment, Many Gaps:** The final Alignment Length is 360 columns, containing an extremely high number of **723 total gaps** (Listing 2). This demonstrates that the algorithm had to introduce extensive gapping to force these disparate sequences into any kind of alignment structure.
- **Alignment Score:** The Sum-of-Pairs Score is **-2039.0** (Listing 2), a strongly negative value. This clearly indicates a very poor quality alignment, dominated by mismatches and the high number of gap penalties incurred.

In conclusion, when applied to these unrelated sequences, the Center Star MSA tool produces an alignment as programmed, but the resulting statistics (very low identity, numerous gaps, and highly negative scores) correctly reflect the profound lack of biological or evolutionary relationship between them.

6 User Interface

6.1 GUI Components

The graphical user interface (GUI) was built using PyQt5 and includes the following features:

- Sequence management (add/remove/load)
- Parameter configuration panel
- Tabbed results display:
  - Alignment results and statistics
  - Center election matrix
  - Pairwise identity/distance matrices
- Export functionality for all results

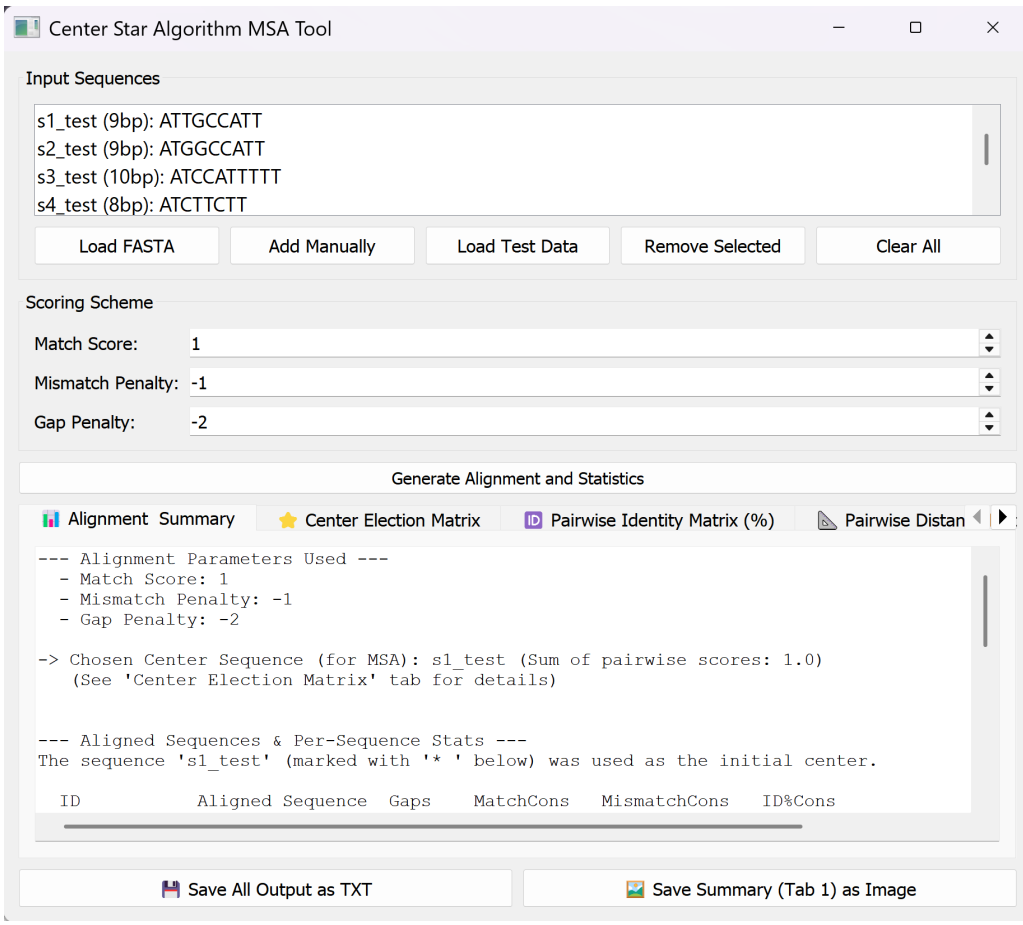

Figure 8: Program graphical user interface showing alignment results